**EX. NO : 6 Concept Learning & Design a Learning system- Find-S Algorithm**

**Date :22.09.2025**

---

**Aim:**

To implement the Find-S algorithm and Candidate Elimination algorithm in Python for concept learning, and to identify the most specific and most general hypotheses consistent with training data.

**Procedure:**

1. **Understand the problem:**
   Concept learning involves finding a hypothesis that fits positive training examples and excludes negative ones.
2. **Prepare the training data:**
   Use a dataset with multiple attributes and a target concept label (e.g., “Yes” or “No”).
3. **Implement Find-S algorithm:**
   - Initialize the hypothesis to the first positive example.
   - Generalize the hypothesis only as needed to accommodate all positive examples by replacing differing attribute values with '?'.
4. **Define helper functions for generalization and specialization:**
   - Check if one hypothesis is more general than another.
   - Generalize the specific boundary (S) to include new positive examples.
   - Specialize the general boundary (G) to exclude negative examples.
5. **Implement Candidate Elimination:**
   - Initialize S to the first positive example and G to the maximally general hypothesis.
   - For each example, update S and G boundaries accordingly:
     - Generalize S when a positive example is encountered.
     - Specialize G when a negative example is encountered.
6. **Extract and print the final specific (S) and general (G) hypotheses.**

**Program Coding:**

```
def train_find_s(data):
    # Initialize hypothesis with the first positive example
    for example in data:
        if example[-1] == "Yes":
            hypothesis = example[:-1]
            break
    # Compare other positive examples
    for example in data:
        if example[-1] == "Yes":
            for i in range(len(hypothesis)):
                if hypothesis[i] != example[i]:
                    hypothesis[i] = "?"
    return hypothesis
# Training data (attribute1, attribute2, ..., PlayTennis)
dataset = [
    ["Sunny", "Warm", "Normal", "Strong", "Warm", "Same", "Yes"],
    ["Sunny", "Warm", "High", "Strong", "Warm", "Same", "Yes"],
    ["Rainy", "Cold", "High", "Strong", "Warm", "Change", "No"],
    ["Sunny", "Warm", "High", "Strong", "Cool", "Change", "Yes"]
]
# Run Find-S algorithm
final_hypothesis = train_find_s(dataset)
# Output the final hypothesis
print("Final Hypothesis:")
```

```
print(final_hypothesis)
```

Final Hypothesis:

```
['Sunny', 'Warm', '?', 'Strong', '?', '?']
def more_general(h1, h2):
    """Check if h1 is more general than h2."""
    more_general_parts = []
    for x, y in zip(h1, h2):
        mg = x == '?' or (x != '0' and (x == y or y == '0'))
        more_general_parts.append(mg)
    return all(more_general_parts)
def generalize_S(example, S):
    """Generalize S to include the positive example."""
    new_S = list(S)
    for i in range(len(S)):
        if S[i] != example[i]:
            new_S[i] = '?'
    return tuple(new_S)


def specialize_G(example, G, S):
    """Specialize G to exclude the negative example."""
    new_G = set()
    for g in G:
        if not more_general(g, example):
            continue
```

```
        for i in range(len(g)):
            if g[i] == '?':
                for val in attributes[i]:
                    if example[i] != val:
                        new_hypothesis = list(g)
                        new_hypothesis[i] = val
                        new_G.add(tuple(new_hypothesis))
            elif g[i] != example[i]:
                continue
    # Remove hypotheses that are more specific than S or not general enough
    return {h for h in new_G if more_general(h, S)}

# Training dataset: [attributes..., class]
dataset = [
    ["Sunny", "Warm", "Normal", "Strong", "Warm", "Same", "Yes"],
    ["Sunny", "Warm", "High", "Strong", "Warm", "Same", "Yes"],
    ["Rainy", "Cold", "High", "Strong", "Warm", "Change", "No"],
    ["Sunny", "Warm", "High", "Weak", "Cool", "Change", "Yes"]
]

# Extract attributes values
attributes = [set() for _ in range(len(dataset[0]) - 1)]
for row in dataset:
    for i, val in enumerate(row[:-1]):
```

```
        attributes[i].add(val)


# Initial boundaries
num_attributes = len(dataset[0]) - 1
S = tuple(dataset[0][:-1])  # First positive example
G = {tuple(['?'] * num_attributes)}


# Candidate Elimination
for row in dataset:
    x, label = row[:-1], row[-1]
    if label == "Yes":  # Positive example
        # Remove from G anything inconsistent with x
        G = {g for g in G if all(g[i] == x[i] or g[i] == '?' for i in range(num_attributes))}
        S = generalize_S(x, S)
    else:  # Negative example
        # Remove from S anything inconsistent with x
        G =specialize_G(x, G, S)
# Final Hypotheses
print("Final Specific Hypothesis S:")
print(S)
print("\nFinal General Hypotheses G:")
for g in G:
    print(g)
```

**Output:**

Final Specific Hypothesis S:

('Sunny', 'Warm', '?', '?', '?', '?')

Final General Hypotheses G:

('Sunny', 'Warm', '?', '?', '?', '?')

**Result:**

Thus, the Find-S and Candidate Elimination algorithms were successfully implemente